

Registro de las sesiones en la academia

Pasos a realizar en la academia para almacenar las sesiones que se agendan en el calendario. Documento técnico de implementación.

1 Qué se va a montar

Cada vez que se agenda una sesión en el calendario, este la envía a la academia, que la guarda. También se envía cuando la sesión se cancela o se reagenda. Con eso la academia puede responder por su cuenta cuántas sesiones da cada profesor al mes, cuál es su media o qué uso real tiene el servicio, sin depender del calendario para nada.

El transporte es el endpoint GraphQL que ya existe, con una mutación nueva autenticada por `X-API-Key`, igual que las de usuarios. El equivalente más parecido a lo que hay que montar es `syncBillingOrder`: mismo patrón de payload y misma idea de sincronizar algo que vive en otro sistema.

Se envía en vez de leerse, y esto es lo que lo decide

La alternativa era que la academia leyera el dato del calendario y no lo guardara, para no duplicarlo. No sirve: el calendario mueve del orden de 500 agendas al día y en algún momento habrá que limpiar reservas viejas o la tabla no para de crecer. Un histórico académico no puede colgar de algo que está previsto borrar. Duplicar es el precio de que sobreviva a esa limpieza.

2 Datos acordados

Se manda el profesor, el nombre del evento y cuándo fue. Es lo justo para auditar el uso del servicio, que es para lo que se pidió.

CAMPO	TIPO	QUÉ ES
<code>reservation_id</code>	String	Id de la reserva en el calendario. Clave del upsert.
<code>status</code>	Enum	<code>CONFIRMED</code> o <code>CANCELLED</code> : GraphQL recibe los enums por su nombre.
<code>academy_id</code>	Int?	La <code>Academy</code> . Puede llegar <code>null</code> — ver punto 4.
<code>professor_email</code>	String	En minúsculas. Empareja con <code>Professor.user.email</code> .
<code>student_lms_id</code>	Int?	El <code>UserProfile.id</code> , del token del iframe. <code>null</code> si la reserva no vino embebida.
<code>student_email</code>	String	En minúsculas. Respaldo del anterior.
<code>event_type_name</code>	String	Ej. «Clase 1 a 1 de inglés».
<code>starts_at</code>	DateTime	ISO-8601 en UTC .
<code>ends_at</code>	DateTime	ISO-8601 en UTC. Va por el feedback (punto 4), no por el recuento.

Lo que no se manda, aunque el calendario lo tenga: los nombres del profesor y del alumno —la academia ya los tiene registrados, y sus datos personales no tienen por qué estar en dos sitios—, el id del tipo de evento, el teléfono, la zona horaria, la URL del Meet, la hora en que se reservó y la duración, que se saca restando.

3 Pasos a realizar

PASO 1 El modelo

Va en `calendars`, pero como modelo aparte de `CalendarEvent`: eso es la agenda propia de la academia —masterclases, recurrencias— y esto es el registro de una reserva que ya ocurrió en otro sistema. Mezclarlos obligaría a que todo lo que hoy consulta `CalendarEvent` aprendiera a distinguir entre las dos cosas.

```
# calendars/models.py
class CalendarSession(HasTimestamps):
    """Sesión agendada en el calendario externo. Histórico, no agenda."""

    class Status(models.TextChoices):
        CONFIRMED = "confirmed", _("Confirmada")
        CANCELLED = "cancelled", _("Cancelada")

    reservation_id = models.CharField(max_length=64, unique=True, verbose_name=_("Id en el calendario"))
    status = models.CharField(max_length=20, choices=Status.choices, default=Status.CONFIRMED,
                             verbose_name=_("Estado"))

    academy = models.ForeignKey(Academy, on_delete=models.PROTECT, null=True, blank=True,
                                related_name="calendar_sessions", verbose_name=_("Academia"))

    # SET_NULL y no CASCADE: dar de baja a un profesor no puede borrar el
    # histórico de las clases que dio. Lo mismo con el alumno.
```

```

professor = models.ForeignKey(Professor, on_delete=models.SET_NULL, null=True, blank=True,
                             related_name="calendar_sessions", verbose_name=_("Profesor"))
student = models.ForeignKey("accounts.UserProfile", on_delete=models.SET_NULL, null=True,
                            blank=True, related_name="calendar_sessions", verbose_name=_("Alumno"))

# Se guardan tal como llegan aunque las FK resuelvan: si un emparejamiento
# falla, sin esto no hay forma de saber de quién era la sesión.
professor_email = models.EmailField(verbose_name=_("Email del profesor"))
student_email = models.EmailField(blank=True, verbose_name=_("Email del alumno"))

event_type_name = models.CharField(max_length=255, verbose_name=_("Nombre del evento"))
starts_at = models.DateTimeField(verbose_name=_("Inicio (UTC)"))
ends_at = models.DateTimeField(verbose_name=_("Fin (UTC)"))

class Meta:
    verbose_name = _("Sesión del calendario")
    verbose_name_plural = _("Sesiones del calendario")
    ordering = ["-starts_at"]
    indexes = [
        # "sesiones de este profesor en este mes" es LA consulta del dashboard.
        # Sin índice escanea la tabla entera, y esta tabla solo crece.
        models.Index(fields=["professor", "starts_at"]),
        models.Index(fields=["student", "starts_at"]),
        models.Index(fields=["academy", "status", "starts_at"]),
    ]

```

```

kool run django makemigrations calendars
kool run django migrate

```

PASO 2 El input y el payload

Los campos se declaran en `snake_case`; Strawberry los expone en `camelCase` solo, igual que en `order_id` o `academy_id`.

```

# api/types/calendar_session.py
@strawberry.input
class SyncCalendarSessionInput:
    reservation_id: str
    status: CalendarSessionStatus # enum: CONFIRMED | CANCELLED
    professor_email: str
    student_email: str
    event_type_name: str
    starts_at: datetime
    ends_at: datetime
    academy_id: Optional[int] = None
    student_lms_id: Optional[int] = None

@strawberry.type
class SyncCalendarSessionPayload:
    success: bool
    created: bool = False
    errors: Optional[List[str]] = None

```

PASO 3 La mutación

Referencia, no receta: la decisión de qué hacer cuando el profesor no está registrado es de la academia (ver punto 4) y aquí va marcada.

```

# api/mutations/calendar_session.py
@strawberry.type
class CalendarSessionMutation:
    @strawberry.mutation(permission_classes=[IsAuthenticatedWithAPIKey])
    def sync_calendar_session(self, input: SyncCalendarSessionInput) -> SyncCalendarSessionPayload:
        professor_email = normalize_email(input.professor_email)
        professor = Professor.objects.filter(user_email=professor_email).first()
        if not professor:
            # DECISIÓN DE LA ACADEMIA. Con success=False el calendario reintenta y
            # avisa, y la sesión entra sola en cuanto se dé de alta al profesor.
            # La otra opción es guardarla con professor=None y el aviso en errors.
            return SyncCalendarSessionPayload(
                success=False, errors=[f"Professor not registered: {professor_email}"]
            )

        # Por id primero: es el UserProfile.id que la propia academia firmó en el
        # token del iframe. El email es el respaldo de lo que no viene embebido.
        student = None
        if input.student_lms_id:

```

```

        student = UserProfile.objects.filter(pk=input.student_lms_id).first()
        if not student and input.student_email:
            student = UserProfile.objects.filter(email=normalize_email(input.student_email)).first()

        academy = Academy.objects.filter(pk=input.academy_id).first() if input.academy_id else None

    _, created = CalendarSession.objects.update_or_create(
        reservation_id=input.reservation_id, # upsert: reintentar no duplica
        defaults={
            "status": input.status.value,
            "academy": academy,
            "professor": professor,
            "student": student,
            "professor_email": professor_email,
            "student_email": normalize_email(input.student_email) or "",
            "event_type_name": input.event_type_name,
            "starts_at": input.starts_at,
            "ends_at": input.ends_at,
        },
    )
    return SyncCalendarSessionPayload(success=True, created=created)

```

PASO 4 Engancharla al esquema

Una línea, y es la que se olvida: sin ella la mutación no existe y no da ningún error.

```

# api/schema.py
from .mutations.calendar_session import CalendarSessionMutation

@strawberry.type
class Mutation(UserMutation, CalendarSessionMutation):
    pass

```

PASO 5 Crear la API key

Una `APIKey` nueva desde el admin, con un nombre que la identifique («Calendario»). Se comparte con el equipo del calendario por gestor de contraseñas, nunca por mensajería. Conviene una distinta para pruebas y para producción.

PASO 6 Comprobar de extremo a extremo

Con la clave configurada en el calendario y un tipo de evento de prueba marcado:

1. Se agenda una sesión de mentira → aparece una fila con `status: confirmed`.
2. Se cancela → la misma fila pasa a `cancelled`. Si aparece una segunda fila, el upsert no está tomando `reservation_id` como clave.
3. Se reagenda → quedan dos filas: la vieja cancelada y una nueva confirmada.

4 Detalles a tener en cuenta

Las canceladas se guardan, pero no cuentan

Hay que almacenarlas en vez de borrarlas, y dejarlas fuera del recuento de «cuántas sesiones dio este profesor en agosto». Si se cuentan, el número mide clases que nunca ocurrieron y deja de servir para nada.

Si algo falla, `success: false`

GraphQL responde 200 aunque la mutación falle, y el error va en el cuerpo. El calendario mira el cuerpo justo para no dar por bueno un envío que no se guardó: un `success: true` de mentira es la forma silenciosa de perder el dato. Devolver el error no pierde la sesión, porque se reintenta.

El profesor puede no resolverse aunque esté registrado

`Professor.user` es un `OneToOneField(null=True)`: un `Professor` sin `user` no tiene email y no empareja con nada. Y aun teniéndolo, el email que manda el calendario es el corporativo con el que esa persona organiza allí, que no tiene por qué coincidir con el de su `UserProfile`.

La elección entre devolver `success: false` o guardar con `professor=None` corresponde a la academia, porque cambia lo que hace el calendario. Lo que conviene evitar es aceptarla con `success: true` y dejarla huérfana sin rastro.

El alumno se empareja por id, con el email de respaldo

`student_lms_id` es el `UserProfile.id`: el mismo `uid` que la academia ya firma dentro del token del iframe, así que es una búsqueda por clave primaria.

El respaldo por email hace falta porque **los tipos de evento que se registran no son solo los que se embeben**. La casilla que activa el envío está en todos los tipos de evento del calendario y se irá marcando lo que interese medir. En los que no se

embeben nunca habrá `student_lms_id`, y en algunos no habrá ningún alumno de la academia al otro lado. Esas sesiones hay que guardarlas igual, con `student=None`: las métricas por profesor siguen valiendo, que son las que se pidieron.

`academy_id` puede venir null

Desde el calendario no siempre se puede deducir a qué academia pertenece una sesión: el código de escuela que usan las demás integraciones sale del embudo o del lead, y una clase 1 a 1 no tiene ninguno de los dos. Se configura a mano en cada tipo de evento, y en cuanto la academia facilite los ids de sus `Academy` viajará siempre puesto.

Hasta entonces, guardarlo a `null` es correcto. No conviene deducirlo del profesor: `Professor` no tiene relación directa con `Academy` —solo la indirecta por `masters`— y un profesor puede dar clase en más de una.

Qué no hace falta hacer

- No hay que exponer ninguna API de lectura para que el calendario consulte nada.
- No hay que tocar `CalendarEvent` ni nada de lo que hoy lo consulta.
- No hay que sincronizar profesores ni alumnos con el calendario.
- No hay que deduplicar por fecha y profesor: el upsert por `reservation_id` ya lo resuelve.
- No hay que subir el histórico anterior. El recuento arranca el día que se enciende.
- No hay que borrar las sesiones canceladas.

5 Qué queda resuelto en el calendario

Ya está implementado y no requiere ninguna acción por parte de la academia. Se detalla para acotar el alcance del trabajo:

- Interruptor por tipo de evento para decidir qué sesiones se registran, con acciones en lote.
- Envío asíncrono con tres reintentos y espera creciente. Una academia caída **nunca** impide que un alumno reserve su clase.
- Envío del ciclo completo: alta, cancelación, reagendamiento y borrado desde el panel.
- Un barrido periódico que reencola lo que no llegó, distinguiendo el alta de la cancelación.
- Un comando para reenviar un rango concreto, por si una caída dura más que el barrido.
- Estado por reserva visible en el admin: enviada, fallida con enlace al error, pendiente o no aplica.
- El `uid` del alumno, guardado desde el token del iframe en el momento de reservar.

6 Comprobación final

- ☐ Modelo creado y migrado, con los índices por profesor y por fecha.
- ☐ Input y payload declarados, con `success`, `created` y `errors`.
- ☐ Mutación protegida con `IsAuthenticatedWithAPIKey`.
- ☐ Mutación añadida a `class Mutation` en `api/schema.py`.
- ☐ El upsert usa `reservation_id` como clave, no la fecha ni el profesor.
- ☐ Decidido y comunicado qué se devuelve cuando el profesor no está registrado.
- ☐ `APIKey` creada y compartida por canal seguro, una por entorno.
- ☐ Comunicada la URL exacta del endpoint GraphQL de cada entorno.
- ☐ Facilitados los ids de las `Academy`, para rellenar `academy_id`.
- ☐ Prueba de extremo a extremo superada: alta, cancelación sobre la misma fila y reagendamiento.